

Guia de Estudio — Modelos Deep Learning para AMR (Segunda Entrega)

Este documento cubre todos los conceptos teoricos utilizados en los modelos profundos del proyecto: BiGRU + Attention, Multi-Stream BiGRU, y Token BiGRU. Organizado para servir como material de estudio con punteros a las fuentes originales.

1. Redes Neuronales Recurrentes (RNN)

1.1 Motivacion: por que las MLP no sirven para secuencias

Un MLP recibe un vector de tamaño fijo y produce una salida. No tiene concepto de “orden” ni de “memoria”. Si le das los mismos numeros en diferente orden, produce la misma salida (salvo por la posicion en el vector de entrada). Pero muchos datos tienen estructura secuencial: texto, audio, series de tiempo, y — en nuestro caso — secuencias de DNA.

Una RNN resuelve esto procesando la entrada **un elemento a la vez**, manteniendo un **estado oculto** (hidden state) que actua como “memoria” de lo que ha visto hasta ahora.

Fuentes: - Haykin (2009), Capitulo 15: “Recurrent Neural Networks” - Goodfellow et al. (2016), Capitulo 10: “Sequence Modeling: Recurrent and Recursive Nets”

1.2 Vanilla RNN: ecuaciones basicas

En cada paso de tiempo t , la RNN recibe: - x_t : el input actual (un vector) - h_{t-1} : el estado oculto del paso anterior (la “memoria”)

Y produce:

$$\begin{aligned}h_t &= \tanh(W_{hh}h_{t-1} + W_{hx}x_t + b_h) \\y_t &= W_{hy}h_t + b_y\end{aligned}$$

El estado oculto h_t es una funcion del input actual y de todo lo que vio antes (a traves de h_{t-1} , que a su vez depende de h_{t-2} , etc.).

Concepto clave: La misma matriz de pesos W_{hh} se reutiliza en cada paso de tiempo. Esto se llama **weight sharing** y es lo que permite procesar secuencias de longitud variable.

Fuente: Haykin (2009), Cap. 15.1-15.2; Goodfellow et al. (2016), Cap. 10.1-10.2

1.3 Backpropagation Through Time (BPTT)

Para entrenar una RNN, se “desenrolla” la red en el tiempo — cada paso temporal se trata como una capa de una red profunda. Luego se aplica backpropagation normal sobre esta red desenrollada.

El problema: si la secuencia tiene T pasos, los gradientes deben fluir a traves de T multiplicaciones por W_{hh} . Esto causa dos problemas:

1. **Gradientes explosivos (exploding gradients):** Si los eigenvalores de W_{hh} son mayores que 1, los gradientes crecen exponencialmente. La red se vuelve inestable.
2. **Gradientes que se desvanecen (vanishing gradients):** Si los eigenvalores son menores que 1, los gradientes se encojen exponencialmente. La red “olvida” lo que vio al inicio de la secuencia.

Fuentes: - Haykin (2009), Cap. 15.3: “Backpropagation Through Time” - Pascanu et al. (2013): “On the Difficulty of Training Recurrent Neural Networks” — analisis formal del problema - Goodfellow et al. (2016), Cap. 10.7: “The Challenge of Long-Term Dependencies”

1.4 Gradient Clipping

Solucion parcial al problema de gradientes explosivos. Antes de actualizar los pesos, se calcula la norma L_2 del vector de gradientes. Si excede un umbral (`max_grad_norm`), se reescala:

$$\text{si } \|g\|_2 > \text{max_grad_norm}, \quad g \leftarrow g \cdot \frac{\text{max_grad_norm}}{\|g\|_2}$$

En el proyecto usamos `max_grad_norm=1.0`. Esto limita la magnitud de cada paso de actualizacion, evitando que un solo batch con gradientes anomalos destruya los pesos aprendidos.

Nota: Gradient clipping NO resuelve el problema de gradientes que se desvanecen. Solo previene la explosion. Para el desvanecimiento, se necesitan arquitecturas con compuertas (GRU, LSTM).

Fuentes: - Pascanu et al. (2013), Seccion 4.1 - PyTorch: `torch.nn.utils.clip_grad_norm_()`

2. GRU (Gated Recurrent Unit)

2.1 Motivacion

La GRU fue propuesta por Cho et al. (2014) como una alternativa mas simple a la LSTM para resolver el problema de dependencias a largo plazo. La idea central: usar **compuertas** (gates) que controlen que informacion del pasado conservar y cual descartar.

2.2 Ecuaciones de la GRU

En cada paso t :

Compuerta de reinicio (reset gate):

$$r_t = \sigma(W_r[h_{t-1}, x_t])$$

Decide cuanto del estado anterior “olvidar” al calcular el candidato.

Compuerta de actualizacion (update gate):

$$z_t = \sigma(W_z[h_{t-1}, x_t])$$

Decide cuanto del nuevo candidato usar vs cuanto del estado anterior conservar.

Estado candidato:

$$\tilde{h}_t = \tanh(W[h_t \odot h_{t-1}, x_t])$$

Propuesta de nuevo estado, usando solo la parte “relevante” del estado anterior (filtrada por r_t).

Estado final:

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

Interpolacion entre el estado anterior y el candidato. Si $z_t \approx 0$, se conserva el estado anterior (memoria a largo plazo). Si $z_t \approx 1$, se adopta el candidato (actualizar con informacion nueva).

2.3 Por que resuelve los gradientes que se desvanecen

La ecuacion $h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$ crea un **atajo lineal** (linear shortcut) entre h_{t-1} y h_t . Cuando $z_t \approx 0$, el gradiente fluye directamente de h_t a h_{t-1} sin multiplicaciones por matrices de pesos. Esto permite que la informacion persista a traves de muchos pasos de tiempo sin degradarse.

Es analogo a las conexiones residuales (skip connections) en ResNets — la red puede aprender a “no modificar” el estado oculto en pasos donde no hay informacion relevante.

2.4 GRU vs LSTM

Aspecto	GRU	LSTM
Compuertas	2 (reset, update)	3 (input, forget, output)
Estado	Solo h_t	h_t (hidden) + c_t (cell state)
Parametros	Menos ($\sim 25\%$ menos que LSTM equivalente)	Mas
Rendimiento	Similar en la mayoria de tareas	Similar, a veces mejor en secuencias muy largas
Velocidad	Mas rapido (menos operaciones por paso)	Mas lento

En el proyecto elegimos GRU siguiendo a [Lugo21], donde funciona bien para secuencias genómicas.

Fuentes: - Cho et al. (2014): “Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation” — articulo original de la GRU - Goodfellow et al. (2016), Cap. 10.10: “Gated RNNs” - Haykin (2009), Cap. 15 (cubre LSTM; GRU es posterior al libro pero los principios aplican)

3. Bidireccionalidad (BiRNN / BiGRU)

3.1 Concepto

Una RNN unidireccional procesa la secuencia de izquierda a derecha: en cada paso t , solo tiene informacion de $x_1, \dots, x_t$. Pero en muchas tareas, el contexto futuro tambien es relevante.

Una BiRNN usa **dos RNNs independientes**: - **Forward RNN**: procesa $x_1, x_2, \dots, x_T$ y produce $\vec{h}_1, \vec{h}_2, \dots, \vec{h}_T$. - **Backward RNN**: procesa $x_T, x_{T-1}, \dots, x_1$ y produce $\overleftarrow{h}_1, \overleftarrow{h}_2, \dots, \overleftarrow{h}_T$.

El estado oculto final en cada posicion es la **concatenacion** de ambas direcciones:

$$h_t = [\vec{h}_t; \overleftarrow{h}_t]$$

Si cada GRU tiene hidden_size=128, el estado concatenado tiene dimension $128 \times 2 = 256$.

3.2 Por que es util para genomas

Una secuencia de DNA no tiene un “inicio” y “final” biologicamente privilegiados — los contigs de un ensamblaje pueden estar en cualquier orden y orientacion. Procesar en ambas direcciones asegura que cada posicion tenga contexto de sus vecinos en ambos sentidos, sin importar la orientacion arbitraria del contig.

3.3 En PyTorch

```
self.gru = nn.GRU(
    input_size=64,          # dimension del input por timestep
    hidden_size=128,        # dimension del estado oculto por direccion
    num_layers=2,           # capas apiladas
    batch_first=True,       # shape [batch, seq_len, features]
    bidirectional=True,     # activa la BiGRU
    dropout=0.3,            # dropout entre capas (requiere num_layers >= 2)
)
# Output shape: [batch, seq_len, 256] (128 forward + 128 backward)
```

Fuentes: - Schuster & Paliwal (1997): “Bidirectional Recurrent Neural Networks” — artículo original - Graves (2012): “Supervised Sequence Labelling with Recurrent Neural Networks” — contexto practico - Goodfellow et al. (2016), Cap. 10.3: “Bidirectional RNNs”

4. Mecanismo de Atencion

4.1 Motivacion: el problema del cuello de botella

Sin atencion, una RNN bidireccional produce T estados ocultos (uno por timestep). Para clasificacion, necesitamos comprimir toda esa secuencia en un solo vector. La solucion naive es tomar el ultimo estado oculto, pero esto fuerza toda la informacion de la secuencia a pasar por un unico vector de dimension fija — un cuello de botella.

La atencion resuelve esto: en lugar de usar solo el ultimo estado, calcula una **suma ponderada** de todos los estados ocultos, donde los pesos indican cuales son mas relevantes para la tarea.

4.2 Atencion aditiva (Bahdanau)

Propuesta por Bahdanau, Cho & Bengio (2015) para traduccion automatica. Las ecuaciones:

Paso 1 — Calcular “energia” de cada timestep:

$$e_t = v_a^\top \tanh(W_a h_t)$$

W_a proyecta cada estado oculto a un espacio de atencion, $\tanh$ introduce no-linealidad, y v_a produce un score escalar. La energia indica “cuanta informacion relevante contiene el timestep t ”.

Paso 2 — Normalizar con softmax:

$$\alpha_t = \frac{\exp(e_t)}{\sum_j \exp(e_j)}$$

Los pesos α_t forman una distribucion de probabilidad sobre la secuencia. Suman 1 y compiten entre si: si un timestep tiene energia alta, “roba” atencion de los demas.

Paso 3 — Suma ponderada (vector de contexto):

$$\text{context} = \sum_t \alpha_t h_t$$

El vector de contexto es un promedio ponderado de todos los estados ocultos, donde los pesos reflejan la relevancia de cada posicion.

4.3 Atencion multiplicativa (Luong)

Alternativa propuesta por Luong, Pham & Manning (2015). En lugar de una red neuronal para calcular la energia, usa un producto escalar (dot product) o un producto bilineal:

- **Dot:** $e_t = h_t^\top h_s$ (solo funciona si las dimensiones coinciden)
- **General:** $e_t = h_t^\top W_a h_s$ (mas flexible)

Es computacionalmente mas eficiente que la aditiva. En la practica, ambos tipos dan resultados similares.

Nota del proyecto: Aunque [Lugo21] describe usar atencion tipo Luong, la implementacion del proyecto usa atencion aditiva (Bahdanau) porque es mas didactica y explicita.

4.4 Atencion como interpretabilidad

Los pesos de atencion α_t indican **donde mira el modelo**. En nuestro proyecto: - Para la BiGRU con histogramas: los pesos muestran que bins de k-meros son mas informativos (se encontro que $k = 3$ concentra 86.77% de la atencion) - Para el Token BiGRU: los pesos indican que posiciones del genoma (muestreadas) son relevantes para la prediccion de resistencia

Esto convierte a la atencion en una herramienta de interpretabilidad — no solo mejora el rendimiento sino que permite entender que aprende el modelo.

Fuentes: - Bahdanau, Cho & Bengio (2015): “Neural Machine Translation by Jointly Learning to Align and Translate” - Luong, Pham & Manning (2015): “Effective Approaches to Attention-based Neural Machine Translation” - Goodfellow et al. (2016), Cap. 12.4.5 (atencion en contexto de deep learning)

5. Embeddings

5.1 Concepto: de discreto a continuo

Un embedding mapea un indice entero (una categoria discreta) a un vector denso de numeros reales en un espacio continuo de dimension fija. En lugar de usar representaciones sparse como one-hot encoding (un vector de N dimensiones con un solo 1), el embedding comprime la informacion en un vector pequeno (e.g., 49 o 64 dimensiones) donde las relaciones entre categorias se aprenden durante el entrenamiento.

Indice 27 → [0.12, -0.45, 0.78, ..., 0.33] (vector de 64 dims)
Indice 28 → [0.14, -0.43, 0.76, ..., 0.31] (cerca si son similares)

La distancia entre vectores de embedding refleja la “similitud” aprendida. Categorias que aparecen en contextos similares durante el entrenamiento terminan con embeddings cercanos.

5.2 nn.Embedding en PyTorch

```
self.embedding = nn.Embedding(  
    num_embeddings=257,      # tamaño del vocabulario  
    embedding_dim=64,       # dimension del vector de salida  
    padding_idx=256,        # este indice siempre se mapea al vector cero  
)
```

Internamente, nn.Embedding es simplemente una **tabla de lookup** — una matriz de forma [num_embeddings, embedding_dim]. El forward pass es un indexado de la tabla, no una multiplicacion. Los gradientes actualizan solo las filas accedidas en cada batch.

padding_idx asegura que el token de padding se mapee al vector cero y no reciba gradientes, para que no contamine la representacion.

5.3 Embedding de antibioticos

En el proyecto, cada antibiotico se representa como un indice entero (0-89). El embedding mapea este indice a un vector de 49 dimensiones. La dimension 49 se eligio con la heuristica $\min(50, (n_{\text{antibiotics}} // 2) + 1)$ con $n_{\text{antibiotics}} = 96$.

El embedding permite que el modelo aprenda relaciones entre antibioticos: antibioticos de la misma familia (e.g., carbapenems) deberian terminar con embeddings cercanos si confieren patrones de resistencia similares.

5.4 Embedding de k-meros (Token BiGRU)

En el Token BiGRU, cada k-mero (secuencia de 4 bases de DNA) se mapea a un ID entero (0-255) usando el hash de 2 bits. El embedding aprende una representacion densa de 64 dimensiones para cada uno de los 256 posibles 4-meros.

La hipotesis es que k-meros biologicamente similares (e.g., que difieren en una sola base, como ACGT y ACGA) desarrollan embeddings cercanos durante el entrenamiento, capturando relaciones semanticas analogas a las de word embeddings en NLP.

Fuentes: - Mikolov et al. (2013): "Efficient Estimation of Word Representations in Vector Space" — Word2Vec, el fundamento de los embeddings aprendidos - Goodfellow et al. (2016), Cap. 12.4: "Distributed Representation" - Haykin (2009), Cap. 7.1: "Representacion del Conocimiento" — marco teorico del mapeo de categorias a espacio continuo

6. Regularizacion

6.1 Que es y por que se necesita

La regularizacion es cualquier tecnica que reduce el error de generalizacion (performance en datos no vistos) a costa de aumentar ligeramente el error de entrenamiento. Es necesaria cuando el modelo tiene suficiente capacidad para memorizar el conjunto de entrenamiento, lo que no garantiza que generalice.

El **dilema sesgo-varianza** (bias-variance tradeoff) es central: un modelo con poca capacidad tiene sesgo alto (no puede aprender los patrones), uno con mucha capacidad tiene varianza alta (memoriza ruido). La regularizacion reduce la varianza.

Fuente: Haykin (2009), Cap. 4.11: "Aproximaciones de la superficie de error"; Goodfellow et al. (2016), Cap. 7

6.2 Dropout

Propuesto por Srivastava et al. (2014). Durante el entrenamiento, cada neurona se "apaga" (se pone a cero) con probabilidad p en cada forward pass. Esto tiene varios efectos:

1. **Evita co-adaptacion:** Las neuronas no pueden depender de neuronas especificas, forzando redundancia en la representacion.
2. **Ensemble implicito:** Cada forward pass entrena una "sub-red" diferente. El modelo final es un ensemble de aproximadamente 2^N sub-redes.
3. **En inferencia (eval):** NO se aplica dropout. Las activaciones se escalan por $(1 - p)$ para compensar (en PyTorch esto se hace automaticamente).

En el proyecto: - Dropout (0.3) en el clasificador (todas las arquitecturas) - dropout=0.3 en nn.GRU con 2 capas (Token BiGRU v2) — se aplica **entre** capas, no dentro de una capa

Fuente: Srivastava et al. (2014): "Dropout: A Simple Way to Prevent Neural Networks from Overfitting"

6.3 Dropout recurrente (entre capas de GRU)

En PyTorch, el parametro dropout de nn.GRU aplica dropout a la **salida** de cada capa recurrente excepto la ultima. Con num_layers=2:

Input → GRU Layer 1 → Dropout(0.3) → GRU Layer 2 → Output

No aplica dropout dentro de las conexiones recurrentes de una misma capa (eso seria “variational dropout”, propuesto por Gal & Ghahramani 2016, y no es lo que implementa PyTorch por defecto).

Con num_layers=1, el parametro dropout se ignora silenciosamente — es por eso que el Token BiGRU necesita 2 capas para habilitar el dropout recurrente.

6.4 Regularizacion L2 (Weight Decay)

Agrega un termino a la funcion de perdida que penaliza los pesos grandes:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{BCE}} + \lambda \sum_i w_i^2$$

Esto “empuja” todos los pesos hacia cero, forzando al modelo a encontrar soluciones con pesos pequenos. Funciones con pesos pequenos tienden a ser mas suaves y generalizar mejor.

En la practica, PyTorch implementa L_2 como weight_decay en el optimizador, que suma λw al gradiente antes de la actualizacion.

6.5 Adam vs AdamW: diferencia en weight decay

- **Adam + weight_decay:** Suma λw al gradiente *antes* del escalado adaptativo. Los parametros con gradientes grandes reciben mas L_2 — no es lo ideal.
- **AdamW (Loshchilov & Hutter, 2019):** Aplica el decaimiento de pesos *despues* del escalado adaptativo, desacoplado del gradiente. Mas correcto teoricamente.

Con weight_decay=1e-4 (pequeno), la diferencia practica es minima. Para valores mas grandes, AdamW es preferible.

Fuentes: - Goodfellow et al. (2016), Cap. 7.1: “Parameter Norm Penalties” - Haykin (2009), Cap. 4.14: “Regularizacion” - Loshchilov & Hutter (2019): “Decoupled Weight Decay Regularization” — propuesta de AdamW

6.6 Early Stopping

Monitorear la perdida en validacion durante el entrenamiento. Si no mejora durante patience epocas consecutivas, detener el entrenamiento. Se guarda el checkpoint del modelo con mejor metrica (en nuestro caso, mejor val F1).

Es una forma de regularizacion porque limita la capacidad efectiva del modelo: le permite aprender patrones generales (primeras epocas) pero lo detiene antes de que memorice el ruido (epocas posteriores).

Fuente: Haykin (2009), Cap. 4.13: “Validacion cruzada”

7. Funcion de Perdida y Desbalance de Clases

7.1 Binary Cross-Entropy (BCE)

Para clasificacion binaria, la funcion de perdida estandar es:

$$\text{BCE}(y, p) = -[y \log(p) + (1 - y) \log(1 - p)]$$

Donde y es la etiqueta (0 o 1) y p es la probabilidad predicha.

BCEWithLogitsLoss en PyTorch combina una capa sigmoid con BCE en una sola operacion, lo que es numericamente mas estable:

```
criterion = nn.BCEWithLogitsLoss(pos_weight=tensor([1.57]))
```

El modelo produce **logits** (valores sin acotar), no probabilidades. La sigmoid se aplica internamente.

7.2 pos_weight: penalizacion asimetrica

Cuando las clases estan desbalanceadas, un error en la clase minoritaria deberia “costar mas” que un error en la mayoritaria. `pos_weight` multiplica la perdida de los ejemplos positivos:

$$\text{BCE}_{\text{weighted}}(y, p) = -[w_+ y \log(p) + (1 - y) \log(1 - p)]$$

- `pos_weight > 1`: Penaliza mas los falsos negativos → sube recall
- `pos_weight < 1`: Penaliza mas los falsos positivos → sube precision
- `pos_weight = n_neg / n_pos`: Equilibra las clases, con $w_+ = n_{\text{neg}}/n_{\text{pos}}$.

En el proyecto, el `pos_weight` base se calcula como `n_susceptible / n_resistente` y luego se escala por un factor configurable (`pos_weight_scale`).

7.3 Cost-Sensitive Learning

El uso de `pos_weight` implementa lo que se conoce como **aprendizaje sensible al costo** (cost-sensitive learning). La idea: en AMR, un Falso Negativo (no detectar resistencia) es clinicamente mucho mas peligroso que un Falso Positivo (falsa alarma). Escalar el `pos_weight` por 2.5 define matematicamente que omitir un organismo resistente es 2.5 veces mas costoso que una falsa alarma.

Esto conecta con la **Teoria de la Decision de Bayes** (Haykin, Cap. 1.4): el umbral optimo de decision depende de la relacion de costos entre tipos de error.

Fuentes: - King & Zeng (2001): “Logistic Regression in Rare Events Data” - Haykin (2009), Cap. 1.4: Clasificador de Bayes - Goodfellow et al. (2016), Cap. 6.2.2

7.4 Umbral de decision optimo (Threshold Calibration)

El modelo produce probabilidades, pero necesitamos una decision binaria (resistente o no). El umbral por defecto es 0.5, pero no siempre es optimo — especialmente con clases desbalanceadas o `pos_weight` escalado.

En el proyecto, despues del entrenamiento se busca el umbral que **maximiza F1 en validacion**. Luego ese umbral se usa para evaluar en test. Esto es importante porque:

1. Buscar el umbral en test seria **data leakage** (usar informacion de test para tomar decisiones).
2. El umbral optimo puede ser muy diferente de 0.5 (en Token BiGRU v2 fue 0.3787).

8. Optimizacion

8.1 Descenso de Gradiente Estocastico (SGD)

El algoritmo fundamental: actualizar los pesos en la direccion opuesta al gradiente de la perdida:

$$w \leftarrow w - \eta \frac{\partial \mathcal{L}}{\partial w}$$

- **Batch GD:** Calcula el gradiente sobre todo el dataset → lento pero gradiente exacto
- **Stochastic GD (SGD):** Un ejemplo a la vez → rapido pero ruidoso
- **Mini-batch GD:** Grupos de N ejemplos (e.g., 32) → balance entre velocidad y estabilidad

8.2 Adam (Adaptive Moment Estimation)

Propuesto por Kingma & Ba (2015). Combina dos ideas:

1. **Momentum:** Mantiene un promedio movil exponencial del gradiente (primer momento). Esto suaviza las oscilaciones y acelera la convergencia en la direccion consistente.
2. **RMSProp:** Mantiene un promedio movil del gradiente al cuadrado (segundo momento). Esto adapta la tasa de aprendizaje por parametro — parametros con gradientes grandes reciben pasos mas pequenos.

$$\begin{aligned}
 m_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t && \text{primer momento} \\
 v_t &= \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 && \text{segundo momento} \\
 \hat{m}_t &= \frac{m_t}{1 - \beta_1^t} && \text{correccion de sesgo} \\
 \hat{v}_t &= \frac{v_t}{1 - \beta_2^t} && \text{correccion de sesgo} \\
 w &\leftarrow w - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} && \text{actualizacion}
 \end{aligned}$$

Defaults: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$.

Fuente: Kingma & Ba (2015): “Adam: A Method for Stochastic Optimization”

8.3 Learning Rate

La tasa de aprendizaje (lr) es el hiperparametro mas importante. Controla el tamaño de cada paso de actualizacion: - **Muy alto:** El modelo oscila y no converge (o diverge) - **Muy bajo:** El modelo converge muy lentamente, puede quedar atrapado en minimos locales - **Justo:** Convergencia rapida y estable

En el proyecto usamos $lr=0.001$ (default de Adam) para los modelos con histograma, y $lr=0.0005$ para el Token BiGRU (para reducir overfitting con convergencia mas gradual).

Fuentes: - Haykin (2009), Cap. 4.5-4.8 - Goodfellow et al. (2016), Cap. 8: “Optimization for Training Deep Models”

9. Arquitectura Multi-Stream (Fusion Multimodal)

9.1 Motivacion

Los modelos BiGRU v1 y v2 procesan la entrada genomica como una matriz [1024, 3] donde las tres columnas son histogramas de $k=3$, $k=4$ y $k=5$, cada uno paddeado con ceros hasta 1024 posiciones. Problema: el padding domina la entrada. $k=3$ tiene solo 64 valores reales y 960 ceros; $k=4$ tiene 256 reales y 768 ceros. El mecanismo de atencion concentra el 86.77% de su energia en las primeras 64 posiciones (donde las tres columnas tienen datos reales), desaprovechando $k=4$ y $k=5$.

9.2 Solucion: streams independientes

En lugar de una sola BiGRU, usar **tres BiGRUs independientes**, cada una procesando su histograma sin padding:

k=3: [batch, 64, 1] → BiGRU_3(hidden=64) → Attention_3 → context_3 [128]
 k=4: [batch, 256, 1] → BiGRU_4(hidden=64) → Attention_4 → context_4 [128]
 k=5: [batch, 1024, 1] → BiGRU_5(hidden=64) → Attention_5 → context_5 [128]

Los tres vectores de contexto se concatenan (384 dims) junto con el embedding del antibiotico (49 dims) y pasan al clasificador (433 → 128 → 1).

9.3 Late Fusion vs Early Fusion

- **Early Fusion:** Combinar las modalidades al inicio (como la matriz [1024, 3] del BiGRU original). Cada capa procesa la informacion mezclada.
- **Late Fusion:** Extraer representaciones independientes de cada modalidad, y combinarlas solo al final (como el Multi-Stream). Permite que cada stream se especialice.

La late fusion es preferible cuando las modalidades tienen escalas o estructuras diferentes (en nuestro caso, secuencias de 64, 256 y 1024 timesteps).

Fuentes: - Ngiam et al. (2011): “Multimodal Deep Learning” — fusion de representaciones multimodales - Goodfellow et al. (2016), Cap. 15: “Representation Learning” — combinacion de representaciones

10. Tokenizacion y Representacion de Secuencias

10.1 Bag-of-Words vs Secuencia

Dos formas fundamentales de representar datos secuenciales:

- **Bag-of-Words (BoW):** Cuenta la frecuencia de cada elemento, ignorando el orden. En nuestro caso: histograma de k-meros. Simple, robusto, pero pierde toda la informacion posicional.
- **Secuencia:** Preserva el orden de los elementos. En nuestro caso: secuencia de IDs de k-meros. Mas expresivo, pero requiere modelos capaces de procesar secuencias (RNNs, Transformers).

10.2 K-meros y el hash de 2 bits

Un k-mero es una subsecuencia de k bases consecutivas de DNA. Con un alfabeto de 4 bases (A, C, G, T), existen 4^k posibles k-meros.

El **hash de 2 bits** codifica cada base con 2 bits: A=00, C=01, G=10, T=11. Un k-mero de longitud k se codifica como un entero de $2k$ bits. Para $k = 4$, esto da valores en $[0, 255]$.

El **rolling hash** permite actualizar el hash al deslizar la ventana una posicion, sin recalcularlo desde cero: se desplaza 2 bits a la izquierda, se agrega la nueva base, y se aplica una mascara para mantener solo $2k$ bits. Complejidad: $O(1)$ por posicion, $O(n)$ para todo el genoma.

Fuente: Compeau & Pevzner (2014), Cap. 9: codificacion 2-bit para k-meros

10.3 Subsampling uniforme

Un genoma bacteriano tipico tiene aproximadamente 4-5 millones de bases, generando millones de k-meros. Ninguna RNN puede procesar secuencias tan largas. El subsampling uniforme selecciona N posiciones equidistantes:

```
indices = numpy.linspace(0, total - 1, max_len, dtype=int)
```

Con $\text{max_len}=4096$ sobre un genoma de 4M bp, se toma un k-mero cada $\sim 1,000$ bp. Esto preserva cobertura global del genoma pero pierde localidad — los tokens vecinos en la secuencia de entrada estan separados $\sim 1,000$ bp en el genoma real.

10.4 Limitacion fundamental: cobertura vs posicion

Este es el hallazgo central del proyecto sobre representaciones:

Histograma: Cobertura 100%, posicion 0% → F1 = 0.86

Tokens (4096): Cobertura ~0.1%, posicion diluida → F1 = 0.81

Ideal: Cobertura 100%, posicion 100% → requiere seq_len ~millones

Los genes de resistencia ocupan aproximadamente 0.02% – 0.2% del genoma. Con subsampling uniforme, un gen de 1,500 bp es cubierto por 1-2 tokens — insuficiente para que la BiGRU aprenda su estructura interna. Los histogramas, al ser un censo completo, capturan la “huella dactilar” de estos genes sin importar su ubicacion.

11. Metricas de Evaluacion

11.1 Matriz de Confusion

		Predicho	
		Pos	Neg
Real	Pos	TP	FN
	Neg	FP	TN

- **TP (True Positive):** Resistente, correctamente predicho como resistente
- **TN (True Negative):** Susceptible, correctamente predicho como susceptible
- **FP (False Positive):** Susceptible, incorrectamente predicho como resistente (falsa alarma)
- **FN (False Negative):** Resistente, incorrectamente predicho como susceptible (fallo critico en AMR)

11.2 Precision, Recall, F1

Precision: De todos los que predije como positivos, cuantos realmente lo son?

$$\text{Precision} = \frac{TP}{TP + FP}$$

Alta precision = pocas falsas alarmas.

Recall (Sensibilidad): De todos los positivos reales, cuantos detecte?

$$\text{Recall} = \frac{TP}{TP + FN}$$

Alto recall = pocos casos perdidos. **Metrica prioritaria en AMR** — no queremos dejar escapar bacterias resistentes.

F1-Score: Media armonica de precision y recall:

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Balancea ambas metricas. Es la metrica principal del proyecto (objetivo: $F_1 \geq 0.85$).

11.3 AUC-ROC

La **curva ROC** grafica la tasa de verdaderos positivos (recall) vs la tasa de falsos positivos ($FPR = FP / (FP + TN)$) para todos los umbrales posibles de decision.

El **AUC-ROC** (Area Under the Curve) mide la capacidad de discriminacion del modelo independientemente del umbral: - AUC = 1.0: clasificador perfecto - AUC = 0.5: clasificador aleatorio (diagonal)

Ventaja sobre F1: AUC-ROC no depende del umbral elegido. Mide si el modelo **separa bien** las clases, no si las clasifica correctamente con un umbral especifico.

11.4 Tabla de resultados del proyecto

Modelo	F1	Recall	AUC-ROC	Precision (est.)
MLP (histograma)	0.8600	0.9165	0.9035	0.8100
BiGRU v2 (histograma)	0.8566	0.9032	0.8998	0.8146
Multi-Stream BiGRU	0.8596	0.8950	0.9038	~0.83
Token BiGRU v2 (secuencia)	0.8121	0.9567	0.8190	~0.70

Fuente: Haykin (2009), Cap. 1.4 (teoria de Bayes para umbrales de decision)

12. Metodologia de Entrenamiento

12.1 Mini-batch Training

En lugar de procesar todo el dataset (batch GD, lento) o un ejemplo a la vez (SGD, ruidoso), se procesan grupos de N ejemplos (mini-batches). Con `batch_size=32`:

1. Se baraja el dataset
2. Se toman 32 ejemplos
3. Forward pass → perdida promedio del batch
4. Backward pass → gradientes
5. Actualizacion de pesos
6. Repetir hasta cubrir todo el dataset = 1 epoca

Ventajas: El ruido del mini-batch ayuda a escapar minimos locales (Haykin, Cap. 4.3). GPU procesa batches en paralelo eficientemente.

12.2 Epocas y Convergencia

Una **epoca** es una pasada completa por todo el dataset de entrenamiento. El entrenamiento tipicamente requiere decenas a cientos de epocas para converger.

Se monitorean dos curvas: - **train_loss**: debe bajar monotonicamente (el modelo aprende) - **val_loss**: debe bajar y luego estabilizarse. Si sube mientras train_loss baja → **overfitting**

12.3 Checkpointing

Se guarda una copia del modelo (`state_dict`) cada vez que la metrica de interes (val F1) alcanza un nuevo maximo. Al final del entrenamiento, se usa el mejor checkpoint — no el modelo de la ultima epoca (que puede estar sobreentrenado).

12.4 Reproducibilidad

Para resultados reproducibles, se fijan semillas en todos los generadores aleatorios:

```
random.seed(42)
numpy.random.seed(42)
torch.manual_seed(42)
torch.cuda.manual_seed_all(42)
torch.backends.cudnn.deterministic = True
torch.backends.cudnn.benchmark = False
```

Misma semilla + mismo hardware + mismo código = mismos resultados.

Fuente: Haykin (2009), Cap. 4.4: “Inicialización”

13. Conceptos Específicos del Dominio (AMR)

13.1 Resistencia Antimicrobiana (AMR)

Capacidad de una bacteria de sobrevivir a un antibiótico que normalmente la mataría. Se adquiere por: - **Mutaciones puntuales** en genes diana (ej. mutación en *gyrA* → resistencia a fluoroquinolonas) - **Transferencia horizontal de genes (HGT):** adquisición de genes de resistencia de otras bacterias vía plásmidos, transposones, integrones - **Regulación de la expresión:** bombas de eflujo (efflux pumps), modificación de permeabilidad

13.2 Grupo ESKAPE

Seis patógenos de alta prioridad por su resistencia y relevancia clínica: - **Enterococcus faecium** - **Staphylococcus aureus** - **Klebsiella pneumoniae** - **Acinetobacter baumannii** - **Pseudomonas aeruginosa** - **Enterobacter spp.**

El proyecto usa genomas de 5 de estas 6 especies (falta *Enterobacter*).

13.3 K-meros como representación genómica

Los k-meros capturan la composición local del DNA. Diferentes especies (y diferentes mecanismos de resistencia) tienen perfiles de k-meros distintivos. Un gen de resistencia como *mecA* (resistencia a meticilina en *S. aureus*) contiene combinaciones de k-meros que son raras o ausentes en genomas susceptibles.

El histograma de k-meros actúa como una “huella dactilar” del genoma — pierde la posición pero retiene la composición. Esta representación resultó ser la más efectiva para predecir AMR en nuestro dataset.

13.4 El problema clínico

En un laboratorio clínico, se necesita saber rápidamente si una bacteria es resistente para elegir el antibiótico correcto. Un falso negativo (predecir susceptible cuando es resistente) puede llevar a un tratamiento inefectivo con consecuencias graves para el paciente. Por eso el **recall es la métrica prioritaria** — es mejor una falsa alarma (recetar un antibiótico más potente de lo necesario) que dejar pasar una bacteria resistente.

14. Arquitectura Completa del Proyecto

14.1 Flujo de datos end-to-end

Genoma FASTA (4-5 Mbp de ACGTACGT...)

↓
 Extraccion de k-meros (k=3,4,5)
 ↓
 └─ Histograma: [64 + 256 + 1024 = 1344] dims → normalizar → MLP
 └─ Matriz BiGRU: [1024, 3] → BiGRU + Attention
 └─ Segmentos: [64,1], [256,1], [1024,1] → Multi-Stream BiGRU
 └─ Tokens: [4096] IDs de k=4 (subsamped) → Embedding → Token BiGRU

14.2 Comparativa de arquitecturas

Componente	MLP	BiGRU v2	Multi-Stream	Token BiGRU v2
Input	[1344] float	[1024, 3] float	3x tupla float	[4096] long
Embedding kmer	No	No	No	Si (257, 64)
Embedding antibiotico	Si (49)	Si (49)	Si (49)	Si (49)
Capa recurrente	No	BiGRU(128) x1	3x BiGRU(64) x1	BiGRU(128) x2
Atencion	No	Bahdanau(128)	3x Bahdanau(64)	Bahdanau(128)
Dropout recurrente	No	No	No	Si (0.3)
Dropout clasificador	Si (0.3)	Si (0.3)	Si (0.3)	Si (0.3)
Clasificador	[1393,512,128,1]	[305,128,1]	[433,128,1]	[305,128,1]
Parametros	~710K	177K	~233K	~537K

15. Referencias Completas

Etiqueta	Referencia	Conceptos
[Haykin]	Haykin, S. (2009). <i>Neural Networks and Learning Machines</i> , 3a ed. Pearson.	RNNs (Cap. 15), BPTT (15.3), regularizacion (4.14), generalizacion (4.11), Bayes (1.4), embeddings (7.1)
[Cho14]	Cho, K. et al. (2014). <i>Learning Phrase Representations using RNN Encoder-Decoder</i> . EMNLP.	GRU: compuertas, ecuaciones, motivacion
[Bahdanau15]	Bahdanau, D. et al. (2015). <i>Neural Machine Translation by Jointly Learning to Align and Translate</i> . ICLR.	Atencion aditiva, vector de contexto, alineamiento
[Luong15]	Luong, M. et al. (2015). <i>Effective Approaches to Attention-based Neural Machine Translation</i> . EMNLP.	Atencion multiplicativa, global vs local
[Schuster97]	Schuster, M. & Paliwal, K. (1997). <i>Bidirectional Recurrent Neural Networks</i> . IEEE Trans. Signal Proc.	BiRNN: forward + backward, concatenacion
[Goodfellow16]	Goodfellow, I. et al. (2016). <i>Deep Learning</i> . MIT Press.	RNNs (Cap. 10), regularizacion (7), optimizacion (8), embeddings (12.4)

Etiqueta	Referencia	Conceptos
[Pascanu13]	Pascanu, R. et al. (2013). <i>On the Difficulty of Training Recurrent Neural Networks</i> . ICML.	Gradientes explosivos/vanishing, gradient clipping
[Srivastava14]	Srivastava, N. et al. (2014). <i>Dropout: A Simple Way to Prevent Neural Networks from Overfitting</i> . JMLR.	Dropout: teoria, implementacion, cuanto usar
[Kingma15]	Kingma, D. & Ba, J. (2015). <i>Adam: A Method for Stochastic Optimization</i> . ICLR.	Optimizador Adam: momentos adaptativos
[Mikolov13]	Mikolov, T. et al. (2013). <i>Efficient Estimation of Word Representations in Vector Space</i> . ICLR.	Word2Vec, embeddings aprendidos
[Ngiam11]	Ngiam, J. et al. (2011). <i>Multimodal Deep Learning</i> . ICML.	Fusion multimodal, late vs early fusion
[Lugo21]	Lugo, L. & Barreto-Hernandez, E. (2021). <i>A Recurrent Neural Network approach for whole genome bacteria identification</i> . Applied AI.	Arquitectura base: BiGRU+Attention para genomas, representacion distribuida de k-meros
[Graves12]	Graves, A. (2012). <i>Supervised Sequence Labelling with Recurrent Neural Networks</i> . Springer.	Sequence labeling, BPTT practico, CTC
[Compeau14]	Compeau, P. & Pevzner, P. (2014). <i>Bioinformatics Algorithms</i> . Active Learning Publishers, Cap. 9.	Rolling hash de 2-bit para k-meros
[King20]	King, G. & Zeng, L. (2001). <i>Logistic Regression in Rare Events Data</i> . Political Analysis.	pos_weight, correccion de prior, cost-sensitive learning